

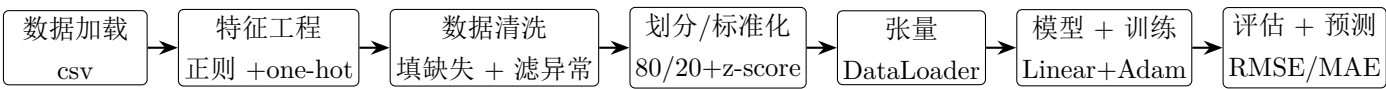
成都二手房单价预测 · 完整流程与数据预处理分析

学习笔记

2026 年 8 月 27 日

1 整体框架（一条流水线）

七个环节首尾相接，数据从「脏 csv」一路变成「能预测的模型」：



步骤	输入	输出	关键动作
1 加载	csv 文件	DataFrame (14838, 16)	<code>read_csv</code>
2 特征工程	原始 df	特征表 ($n, 128$)	正则拆户型/楼层 + one-hot
3 清洗	特征表	干净特征 + 标签	填中位数 + 删异常行
4 划分/标准化	干净数据	训练/测试集	80/20 随机划分 + z-score
5 张量	DataFrame	torch 张量	<code>.values</code> + float32 + DataLoader
6 训练	张量	模型	Linear + MSELoss + Adam, 50 epochs
7 评估	模型	RMSE/MAE	反变换回真实价格 + 新样本预测

2 数据预处理逐行分析（重点）

预处理 = 流水线的第 1-4 步。目标：把「脏 csv」变成「标准化张量」。

2.1 第 1 步 · 特征工程

2.1.1 户型解析：文本拆成 4 个数字（核心）

```
def parse_hu_xing(s):
    def pick(pattern):
        m = re.search(pattern, str(s)) # 正则匹配
        return int(m.group(1)) if m else 0 # 匹配到取捕获数字，否则填 0
    return (pick(r'(\d+)室'), pick(r'(\d+)厅'),
            pick(r'(\d+)厨'), pick(r'(\d+)卫'))
```

逐行要点：

- `re.search(pattern, str(s))`: 在文本中搜索模式。`str(s)` 保证即使输入是 NaN 也不崩。
- 返回 **Match** 对象（或匹配不到时为 None）: 用 `.group(1)` 取第一个括号捕获的数字字符串。
- `int(m.group(1)) if m else 0`: 匹配到就转整数，否则填 0——对应「3 室 2 厅 2 卫」没有厨的情况。
- 同一个 `pick` 函数复用四次，模式只差一个字（室/厅/厨/卫）。

作用:「2 室 1 厅 1 厨 1 卫」这类文本无法直接参与运算，拆成 (2,1,1,1) 四个数值特征后，模型才能用。

2.1.2 楼层解析：位置编码 + 总楼层

```
def parse_floor(s):
    s = str(s)
    if '低' in s: pos = 0
    elif '中' in s: pos = 1
    else:         pos = 2  # 默认按「高」处理
    m = re.search(r'共(\d+)层', s)
    total = int(m.group(1)) if m else None
    return pos, total
```

- 位置编码: 低/中/高是有序类别，直接映射成 0/1/2（有序，用数字没问题，不需要 one-hot）。
- 总楼层:「共 7 层」里的 7，取不到返回 None（交给后面的缺失值处理）。
- 对比: 市辖区/朝向这类无序类别才用 one-hot——这是「有序类别编数字、无序类别 one-hot」的原则。

2.1.3 特征构建：数值 + one-hot

```
def build_features(df):
    feat = pd.DataFrame()
    hu = df[COL['户型']].apply(parse_hu_xing) # 整列应用解析函数
    feat['室'] = [h[0] for h in hu]           # 拆出四个数值列
    ...
    fl = df[COL['楼层']].apply(parse_floor)
    feat['楼层位置'] = [f[0] for f in fl]
    feat['总楼层'] = [f[1] for f in fl]
    feat['建筑面积'] = df[COL['面积']]
    for col in cat_cols:                       # 类别列逐个 one-hot
        filled = df[col].fillna('未知')        # 先填「未知」再编码
        dummies = pd.get_dummies(filled, prefix=col)
        feat = pd.concat([feat, dummies], axis=1)
    return feat
```

- `apply(parse_hu_xing)`: 对整列的每个元素调用解析函数，返回元组序列。
- `[h[0] for h in hu]`: 列表推导拆出元组的第 0 个分量，变成新列。
- one-hot 循环: 5 个类别列（市辖区/朝向/装修/结构/电梯）逐个 `get_dummies` 后横向拼接。
- `fillna('未知')` 先做: 把缺失当一类「未知」，避免 one-hot 后出现整行全 0 的样本。
- 结果: 7 个数值列 + 121 个 one-hot 列 = 128 维特征。

为什么类别用 **one-hot** 而不用数字: 装修「精装 =3、毛坯 =1」会让模型误以为精装大于毛坯（引入不存在的序关系）; one-hot 每类独立一列 0/1，类别之间地位平等。

2.2 第 2 步 · 数据清洗

```
def clean_data(feats_df, label_series):
    feat = feats_df.copy(); y = label_series.copy()
    feat['总楼层'] = pd.to_numeric(feat['总楼层'], errors='coerce') # 字符串 -> NaN
    feat['建筑面积'] = pd.to_numeric(feat['建筑面积'], errors='coerce')
    feat['总楼层'] = feat['总楼层'].fillna(feat['总楼层'].median()) # 填中位数
    feat['建筑面积'] = feat['建筑面积'].fillna(feat['建筑面积'].median())
    mask = (feat['建筑面积'] > 0) & (feat['建筑面积'] <= 500) & (y > 0)
    return feat[mask].reset_index(drop=True), y[mask].reset_index(drop=True)
```

- `pd.to_numeric(..., errors='coerce')`: 真实数据里数值列可能混有「暂无」这类字符串，先把非数字转成 NaN（coerce = 强转，失败变 NaN）。
- `fillna(median)`: 缺失填中位数而不是均值——中位数不受极端值影响，更稳健。
- 异常行过滤 `mask`: 面积 ≤ 0 或 > 500 、单价 ≤ 0 的行是脏数据，整行删除。
- `reset_index(drop=True)`: 过滤后行号会乱，重置成干净的 0..n-1（pandas 按 index 对齐，乱 index 会错位）。
- 为什么过滤异常行: MSE 对离群值敏感（平方放大），脏数据会把参数带偏。

2.3 第 3 步 · 划分与标准化

```
idx = torch.randperm(num_examples).tolist() # 随机打乱索引
n_train = int(0.8 * num_examples)
train_idx, test_idx = idx[:n_train], idx[n_train:]

train_mean = X_raw.iloc[train_idx][NUM_COLS].mean() # 只用训练集统计量
train_std = X_raw.iloc[train_idx][NUM_COLS].std()
y_mean = float(y_raw.iloc[train_idx].mean())
y_std = float(y_raw.iloc[train_idx].std())
```

```
def norm(part):
    out = part.copy()
    out[NUM_COLS] = (out[NUM_COLS] - train_mean) / train_std # z-score
    return out

X_train = norm(X_raw.iloc[train_idx]).reset_index(drop=True)
X_test = norm(X_raw.iloc[test_idx]).reset_index(drop=True)
y_train = (y_raw.iloc[train_idx].values - y_mean) / y_std
y_test = (y_raw.iloc[test_idx].values - y_mean) / y_std
```

- `torch.randperm`: 随机打乱索引后切 80/20, 保证训练/测试分布一致。
- **z-score 标准化**: $(x - \mu)/\sigma$, 把每个数值特征拉到「均值 0、标准差 1」。
- **为什么标准化（核心）**: 面积约 100、室数约 2, 量纲差 50 倍。不标准化时梯度下降在面积方向步子小、在室数方向步子大, 收敛慢甚至震荡; 标准化后各维度同尺度, 收敛快。
- **为什么测试集用训练集的统计量（核心）**: 测试集模拟「上线后的未来数据」, 只能用训练时确定好的变换参数; 用测试集自己的统计量 = 偷看未来 = 信息泄漏。
- **为什么标签也标准化**: 单价约 10^4 量级, 与特征差 4 个数量级, 损失和梯度尺度巨大; 标签 z-score 后损失在 1 附近, 学习率好调。预测时乘回 `y_std` 加 `y_mean` 还原真实价格。

3 一句话总结

预处理四步的本质: 文本转数值（正则）→ 类别转 0/1（one-hot）→ 缺失/异常清理（中位数 + 过滤）→ 统一尺度（z-score）。每一步都在回答同一个问题: 怎么把「脏的真实数据」变成「模型能稳定学习」的形状与尺度。